



OPEN Efficient federated graph aggregation for privacy-preserving GNN-based session recommendation

Jing Lou[✉], Cheng Rong, Hanshen Chen & Daxue Liu

Graph Neural Networks (GNN) have attracted increasing attention due to their efficient performance in recommendation systems. However, applying GNNs in session-based recommendations with emerging federated learning (FL) for a privacy-preserving recommendation is challenging. Firstly, constructing a global graph in a centralized manner is forbidden due to the privacy-preserving constraints of FL. Secondly, local graphs in each device contain minimal information on the global graph, causing the inefficient merging of sub-graphs by aggregating local models. Thirdly, the session data in these separated devices are usually extraordinarily non-Independent and Identically Distributed (non-IID), which harms the model performance. In this paper, we bridge the practical gaps between FL and GNN-based session recommendations for the first time by introducing a novel adaptive federated learning method named Federated Graph Aggregation (FedGA). FedGA is beyond the reach of prior adaptive FL methods by incorporating Divergence Resistant Aggregation (DRA) and Conditional Second-Moment Estimation (C-SME), yielding an efficient aggregator where local models trained by the unseen local graph embedding can be efficiently merged. Thanks to the above-proposed strategies, FedGA optimizes models without being interfered with by the aggressive learning rates generated by existing adaptive methods under extreme non-IIDness. In addition, we perform the theoretical analysis of the proposed method, and the results demonstrate that our method achieves a similar rate of convergence as other adaptive FL methods. We validate our method on both open datasets and real-world production data. Results show that our method obtains state-of-the-art performance compared to existing adaptive FL methods while retaining the comparable performance of the centralized methods.

In recent years, session-based recommendation (SR) methods have attracted increasing attention^{1–22}. These methods predict the next item that a user most probably clicks on or buys without requiring the user/item features. However, different aspects, for example, time, environment, and social relationships, can affect user interests, causing a significant shift in user preferences. Graph Neural Network (GNN) has recently attracted attention for achieving state-of-the-art performance on SR tasks. It has been applied to better capture user preference changes by learning the latent relationships between users and item sequences.

The promising performance of GNN for session-based recommendation is based on a large scale of sensitive data collected from user devices, which raises regulatory challenges for modern recommendation systems to deliver secure and privacy-preserving services. In addition, building efficient and scalable GNNs on large-scale data is challenging. The recent emerging federated learning (FL) is a candidate paradigm for recommendation systems that build models without sharing local data on devices. Furthermore, it naturally provides the personalization ability to the devices by fine-tuning the distributed models using local data. More importantly, applying FL on existing GNNs for session-based recommendation can easily solve scalability issues without building an ad hoc GNN model for large-scale data. In the FL scheme, only dozens of data samples are processed and used to train local models in devices, which can be done in much shorter time with lower computation costs. In short, the FL scheme is urgent under large-scale data for privacy-preserving in session-based recommendations with GNNs. In the rest of this paper, we use *GNN-SR* as the abbreviation for session-based recommendation with GNNs.

However, applying GNNs for such a scenario with FL is still challenging. Firstly, constructing the global graph embedding in a centralized manner is forbidden due to the privacy-preserving constraints of FL. An

College of Intelligent Transportation, Zhejiang Institute of Communications, Hangzhou, China. ✉email: loujing@zjvit.edu.cn

alternative solution is to aggregate encoded local graph embedding constructed with local sessions. However, two-fold challenges lay ahead: 1) local graphs in devices contain minimal relation information, causing inefficient merging of these graphs. 2) There exists an aggregation error issue when directly aggregating local graph embedding. We illustrate this issue in Figure 1. The “local updating” scheme is training a GNN model in a centralized manner, which is supposed to be “correct.” In the “federated updating” scheme, the distributed devices collaboratively train models, and a central server aggregates graph embedding from different devices using FedAvg²³. The embedding matrix in “embedding error” is the difference between the correct and the aggregated embedding. The conventional FedAvg fails to aggregate the graph embedding correctly, since the weights of rarely clicked items (i.e. long-tail items that are only clicked in a small part of devices) are incorrectly divided by the number of devices. However, it is not permitted to observe who and how many users have clicked these items due to the privacy-preserving constraints of federated learning.

Another fundamental challenge is the performance degradation caused by the extreme non-IIDness (non-Independent and Identically Distributed) in real-world, large-scale recommender systems. The user preferences usually diverge a lot and are probably distributed in a power-law tail. Although FL was intentionally designed for such non-IIDness, it still suffers a significant loss in performance when local data in devices differ significantly from each other. To temper the negative impact of non-IIDness, some adaptive methods for FL are proposed^{24,25}. These methods adopt the well-known adaptive methods to optimize deep learning models in FL to improve generalization. However, existing adaptive methods may choose aggressive learning rates under extreme non-IID cases in the FL context, which may poison the performance. Putting the above issues together, the errors when applying FL in GNN-SR are easily magnified. Thus, efficient federated learning methods for these issues are needed.

This paper proposes a federated learning framework for GNN-SR to address the issues mentioned above. Our contributions are summarized as follows:

- We propose FedGA (**F**ederated **G**raph **A**ggregation), a novel adaptive federated optimization method to alleviate performance issues caused by incorrect embedding updating and extreme non-IIDness.
- Two distinct strategies in FedGA, namely **D**ivergence-**R**esistant **A**ggregation (DRA) and **C**onditional **S**econd-**M**oment **E**stimation (C-SME) to eliminate aggressive learning rates in existing adaptive FL methods and hence improve model performance.
- We analyze the FedGA's convergence bound theoretically and obtain the same results as the best-known adaptive methods.
- We conduct extensive experiments on large-scale SR datasets to validate our framework's efficiency under different levels of non-IIDness. The results indicate that our approach outperforms other adaptive FL methods and retains similar performance to locally trained GNN-SR models.

Related work

In this section, we review and compare related GNN-SR methods, FL methods for non-IID data, adaptive FL methods, and FL methods for GNN models.

Session-based Recommendation using GNNs. Incorporating GNNs has become an efficient practice for session-based recommendations. Such methods model the item sequence in a session as a graph structure or embedding to capture items' latent relationships in different sessions. Xu et al.¹⁴ proposed a Graph Contextualized Self-attention (GC-SAN), which combines a self-attention network and GNN to enhance the performance of

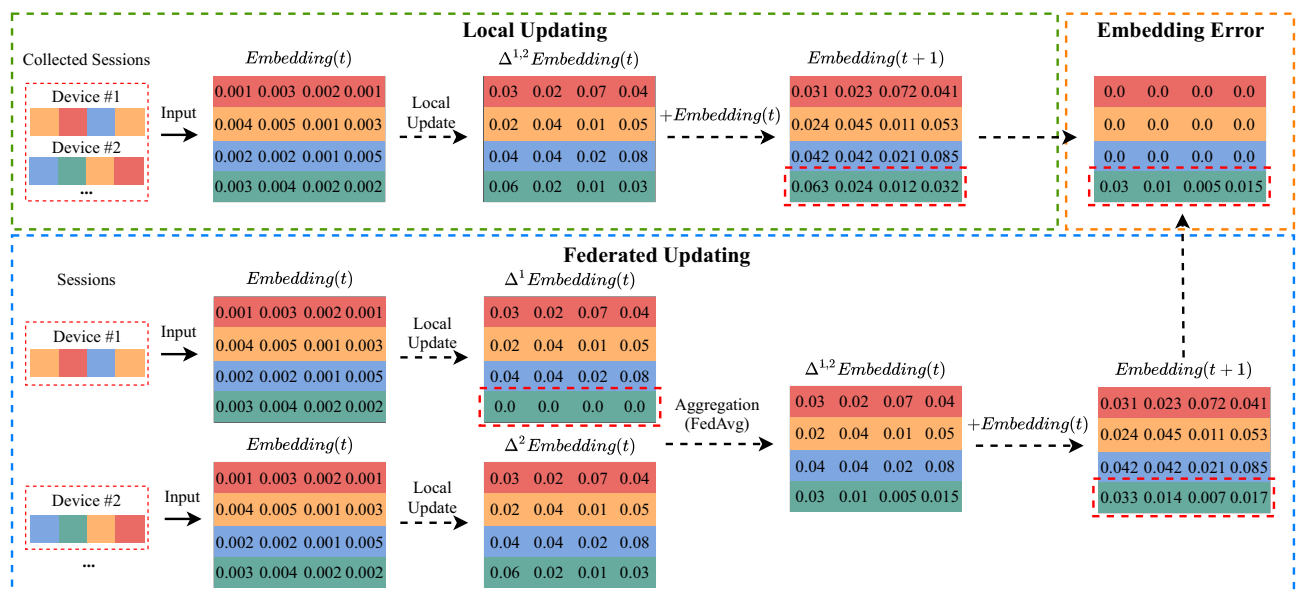


Fig. 1. Incorrect aggregation of graph embedding when applying FL for GNN-SR.

the recommendation. Wu et al.²⁶ converted sequence data of items to a structured direct graph and increased the weights of the latest item, which finally improved the representation of the latest user interests. Qiu et al.¹⁵ proposed a Full Graph Neural Network (FGNN), an end-to-end model for the next-item recommendation in sessions. The main idea of this work is to learn the inherent order of the transition pattern of items. They applied 1) a multiple Weighted Graph Attention (WGAT) layer network to learn and assign appropriate weights to different neighbors in a graph and 2) a read-out function to generate graph-level representation. Song et al.¹⁶ proposed a dynamic graph attention neural network to model dynamic user behaviors and infer influencers according to the context of user interest. Yu et al.¹⁷ proposed the Target Attentive Graph Neural Network (TAGNN), which captures complex item transitions in sessions. Qiu et al.¹⁸ proposed the Global Attributed Graph (GAG), which combines user embedding and a long-term session sequence as the global attribute to improve the performance on long-term user interests.

Federated Learning Methods for non-IID Data. Alleviating the impact of non-IID data across large-scale devices is one of the key tasks for FL. McMahan et al. proposed FedAvg, the first method that claims promising performance for collaborative and privacy-preserving machine learning on non-IID data²³. However, recent studies have observed unstable performance when the data is high-level non-IID. Zhao et al.²⁷ discovered a 55% performance degradation for FedAvg with highly skewed non-IID data. They argued that the degradation is caused by the divergence of model weights and the calculated weight divergence between local models using Earth-Mover Distance (EMD). However, their method requires globally shared data, which usually violates the data-sharing constraints in FL systems. Xie et al.²⁸ proposed a method using multiple centers to better capture the pattern of non-IID data. Briggs et al.²⁹ proposed a clustering approach that evaluates the similarities of different device models and groups them for a more robust aggregation. Yu et al.³⁰ proposed three strategies to improve performance on non-IID data. However, the above methods did not provide theoretical convergence guarantees. Li et al.³¹ gave the first theoretical convergence bounds of FedAvg on non-IID data. Nevertheless, its convergence ability is not proven for non-convex problems. Sahu et al.³² proposed FedProx, a FedAvg variant that applies an L_2 regularizer term in the objective functions of local models, and proved convergence guarantees for both convex and non-convex problems. Similarly, Shoham³³ proposed Federated Curvature, which introduces a penalty term to reduce the impact of non-IIDness. Jeong et al.³⁴ proposed federated augmentation (FAug), a data augment method that collaboratively trains a generative model in devices to generate more local data in devices. The results show a significant improvement in the accuracy of FedAvg with augmented data. Cong et al.³⁵ introduced a federated learning method that relies on a greedy strategy. This strategy gradually searches for models that are partially optimal and combines them to achieve the global model. However, the above FL methods do not apply to both GNNs and session-based recommendations.

Adaptive Federated Optimization. Inspired by successful adaptive optimization methods, Reddi et al.²⁵ proposed a federated version of adaptive optimization methods, namely FedAdagrad, FedAdam, and FedYogi. Furthermore, they analyzed and provided convergence guarantees for non-convex problems. However, in the session-based recommendation, the above adaptive methods may suffer from the fluctuate adaptive learning rates due to the extreme non-IIDness, resulting in unstable performance. Ju et al.³⁶ proposed a federated learning method named AdaFedAdam, which addresses the fairness problem by formulating it as a multi-objective optimization problem, analyzes the performance of Adam optimizer, and proposes an adaptive approach to achieve fair and efficient federated learning with enhanced global model performance. Finally, some adaptive methods fail to obtain stable performance, as the server cannot observe the data size in distributed devices, causing incorrect aggregation on graph embedding.

Federated Learning with GNNs. Jiang et al.³⁷ proposed Feddy, a distributed and secured framework to learn object representations from multi-user graph sequences. However, Feddy requires full participation of all distributed nodes, computes all adjacent information from these nodes, and records the status of the node for each second, which is not efficient and practical in large-scale real-world recommender systems. Sajadmanesh and Gatic-Perez³⁸ developed a GNN learning algorithm that preserves privacy with formal privacy guarantees based on Local Differential Privacy (LDP) to protect node characteristics. However, their method is still under full-node participation and constructs a global graph using all nodes, which is forbidden in common FL scenarios since all nodes are isolated. Zheng et al.³⁹ proposed ASFGNN (Automated Separated-Federated Graph Neural Network) for GNN under non-IID data. However, ASFGNN uses only conventional averaging in model aggregation, which will eventually cause the information loss issue as we mentioned previously, and such an issue is not easy to solve by optimizing hyper-parameters. In addition, ASFGNN uses Bayesian Optimization (BO) to find the best hyperparameters, which is a black-box process, and the performance may not be stable in federated learning when model gradients vary. Wang et al.⁴⁰ proposed two federated learning methods for GNNs under non-IID data. Their methods can generalize into new label domains thanks to self-training strategies. However, their methods have two-fold limitations: 1) They assume that each device has the complete graph in experiments, which is not realistic when the global graph contains a large scale of nodes. 2) They argue that the overall performance is greatly affected by the fraction of overlap between device graphs, which is commonly seen in real-world recommender systems. Wu et al.⁴¹ proposed FedGNN, a federated GNN framework for privacy-preserving recommendations. The authors apply local differential privacy techniques to protect user information and user-item interactions. However, although encrypted user embedding is aggregated in FedGNN, the information loss issue in aggregating graph embedding was not addressed. Wan et al.⁴² proposed a prototype-based approach to tackle the domain shift in federated graph learning. Their method learns generalizable prototypes across clients to align local and global representations, thus improving generalization under distributional heterogeneity. To date, there is no effective federated learning framework for GNNs that directly addresses challenges in session-based recommendation or the embedding aggregation issues specific to GNN-SR under non-IID conditions.

Compared to existing methods, our method does not hold the assumption that each device shall hold the same global graph, and it is flexible to realistic recommender systems under extreme non-IID data.

Preliminaries

In this paper, we focus on federated learning in *cross-device* settings for GNN-SR. In this section, we give the preliminary definitions of FL and GNN-SR.

Session-based Recommendation with GNN. We use similar definitions proposed by Wu et al.²⁶ as a typical example of GNN-SR. Let $\text{mathscr{V}} = \{\nu_1, \nu_2, \dots, \nu_n\}$ be the item set in all sessions, where n is the total number of items. We denote a user session as $s = \{\nu_{s,1}, \nu_{s,2}, \dots, \nu_{s,n_s}\}$, where n_s is the number of items in session s , and $\nu_{s,i} \in \text{mathscr{V}}$. We then define the graph structure of the session s as $\text{mathscr{G}}_s = (\text{mathscr{V}}_s, \text{mathscr{E}}_s)$, where $\text{mathscr{V}}_s$ is the sequence of clicked items, and $\text{mathscr{E}}_s$ is the edge that links two consecutive clicked items in $\text{mathscr{V}}_s$. The session graphs are then converted to the embedding as the input of a GNN model. Finally, the GNN model learns the latent session embedding vectors and predicts the top- k items with the highest probability as the next recommended items. Please note that in different GNN-SR methods, the graph structure and the construction of graph embedding can be different. For example, TAGNN introduces the adjacency matrix $\text{mathscr{A}}$ in the graph structure of sessions, i.e., $\text{mathscr{G}}_s = (\text{mathscr{V}}_s, \text{mathscr{E}}_s, \text{mathscr{A}}_s)$, and introduces pseudo-interaction items in the graph embedding.

Federated Learning. Federated learning is minimizing the objective function of a global model that can be described as:

$$\min_{w \in \mathbb{R}^d} f(w) = \frac{1}{m} \sum_{i=1}^m F_i(w), \quad (1)$$

where $F_i(w)$ is the objective function of the local model in i^{th} device, m is the total number of devices, and d is the model dimension.

Federated Learning on GNN-SR. The typical GNN models for session-based recommendation include the graph embedding layer and the model weights layers (e.g.,^{17,26}). The device models learn and update the weights of the embedding layer and other middle layers locally using only local data, and then send the model parameters (including the embedding layer and middle layers) to the central server. The central server aggregates the collected model gradients using federated averaging:

$$w^{t+1} = \frac{1}{m} \sum_{i=1}^m w_i^t = w^t + \frac{1}{m} \sum_{i=1}^m (w_i^t - w^t) = w^t + \frac{1}{m} \sum_{i=1}^m \Delta w_i^t, \quad (2)$$

where w_i^t includes model parameters of both the embedding layer and middle layers as mentioned earlier.

An intuitive way to train GNN models collaboratively using FL is as follows: 1) building the embedding session locally on devices, 2) training local GNN models using the embedding of the local session, and 3) updating the global model using the parameters collected from the GNN model on the central server. We give an example of GNN-SR under FL in Figure 2. However, as discussed earlier, directly updating the global model using conventional FL (e.g. FedAvg) may result in performance degradation. The current emerging adaptive federated optimization (for example, FedAdam and FedYogi proposed in Reddi et al.²⁵) is a candidate solution for GNN-SR due to adaptive strategies in updating the global model. In the next section, we propose FedGA, a novel adaptive FL method for GNN-SR.

The proposed method

Method overview

We illustrate the main FedGA processes in Algorithms 1 and 2. The central server initializes a neural network model as the global model, sets the maximum number of global epochs T , and initializes the current epoch number $t = 0$. At each global epoch, the central server randomly samples part of the device set S_t from the devices. The sampled devices initialize their local models using the received global model and train them using the corresponding local data. When local training is finished, the sampled devices upload trained model gradients to the central server. Then, the central server calculates the mean of the collected model gradients Δ_t (lines 8 and 9 of Algorithm 1).

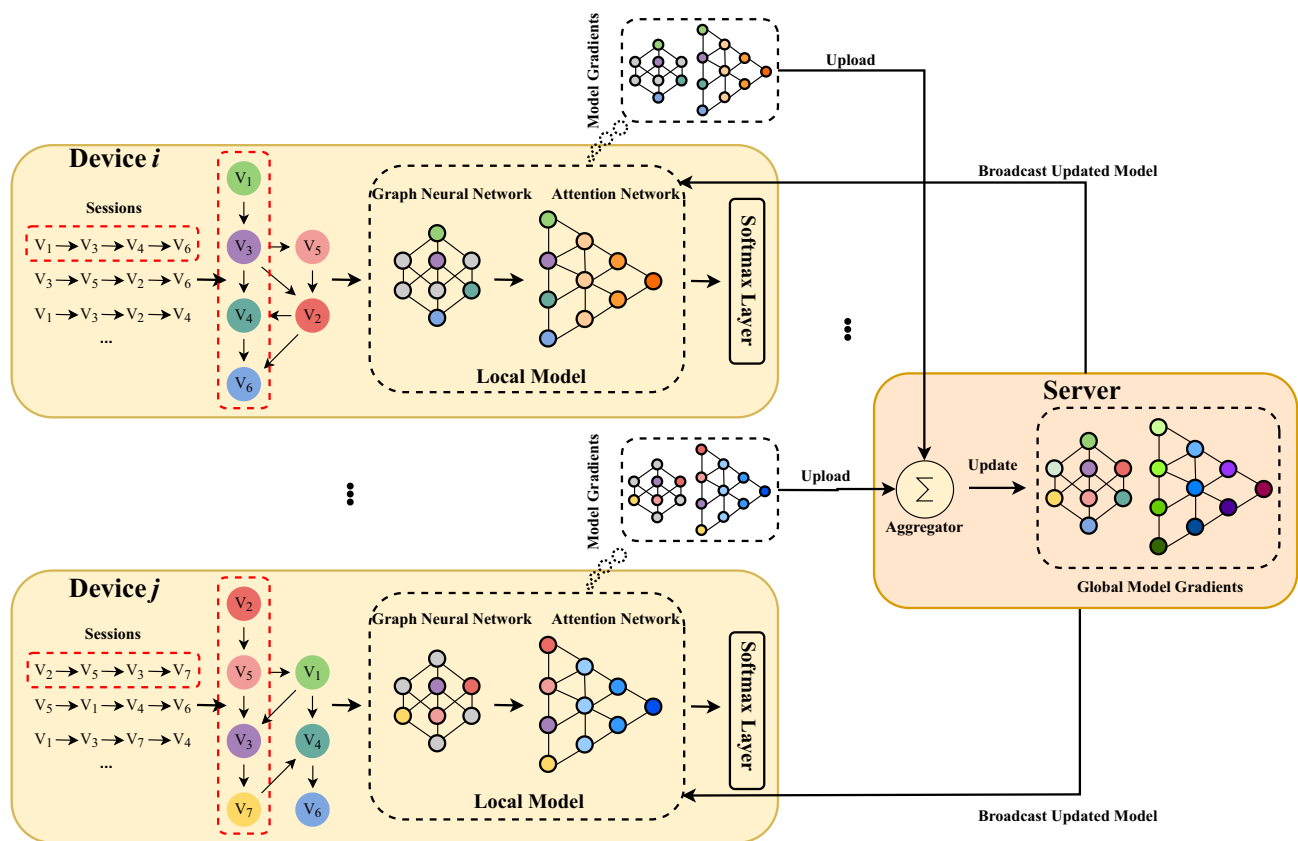


Fig. 2. An example of GNN-SR under the federated learning scheme. We use the GNN model proposed in Wu et al.²⁶ as the example GNN model.

-
- 1 **Input:** Model $F(w^0)$, maximal number of local epoch K , maximal number of global epoch T , global learning rate η_0 , global decay factor γ , local learning rate η_l , control parameters β_1 and β_2 .
 - 2 **Output:** Updated global model weights w .
 - 3 Initialize $v_0 = 0, m_0 = 0$ and initialize w^0 .
 - 4 **for** $t = 1, 2, \dots, T$ **do**
 - 5 Uniformly sample a subset of devices at random: $S_t \subset [m]$.
 - 6 **for each device** $i \in S_t$ **in parallel do**
 - 7 $\Delta w_i^t = \text{LocalUpdate}(w^{t-1}, K, \eta_l)$. $\triangleleft$ *In-device processes*
 - 8 Collect Δw_i^t from devices $i \in S_t$.
 - 9 $\Delta_t = \frac{1}{|S_t|} \sum_{i \in S_t} \Delta w_i^t$.
 - 10 $\Delta'_t = \frac{1}{|S_t|} \sum_{i \in S_t} |\Delta w_i^t|$. $\triangleleft$ *DRA*
 - 11 $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \Delta_t$.
 - 12 Update v_t by **Eq. (3)**. $\triangleleft$ *C-SME*
 - 13 $w^t = w^{t-1} + \frac{\eta_0}{1+\gamma} \frac{m_t}{\tau + \sqrt{v_t}}$, where $\tau = 1e - 6$.
-

Algorithm 1. FedGA - Server

-
- 1 **Input:** Global model w^{t-1} , local update number K , local learning rate η_l .
 - 2 **Notation:** $\mathcal{L}_i(w)$ denotes the local loss function on client i ;
 - 3 $g_{i,k+1} = \nabla \mathcal{L}_i(w_{i,k}^{t-1})$ is the local gradient at step $k+1$.
 - 4 Initialize $w_{i,0}^{t-1} = w^{t-1}$.
 - 5 **for** $k = 0, 1, \dots, K-1$ **do**
 - 6 Compute gradient: $g_{i,k+1} = \nabla \mathcal{L}_i(w_{i,k}^{t-1})$.
 - 7 Update model: $w_{i,k+1}^{t-1} = w_{i,k}^{t-1} - \eta_l g_{i,k+1}$.
 - 8 Compute update: $\Delta w_i^t = w_{i,K}^{t-1} - w^{t-1}$.
 - 9 Send Δw_i^t to server.
-

Algorithm 2. LocalUpdate

The rest of the processes are similar to the procedures of adaptive optimization, i.e., calculating the first- and second-moment estimations of model gradients and using them to update the global model. Firstly, for the first-moment estimation, we introduce Divergence-Resistant Aggregate (DRA) to alleviate the impact of sparse gradients collected from devices (line 10 of Algorithm 1). For the second-moment estimation (SME), we propose Conditional Second Moment Estimation (C-SME), which carefully estimates the second-order moments under three conditions, i.e.,

$$v_t = \begin{cases} v_{t-1} + (1 - \beta_2) \sqrt{(\Delta_t'^2 + v_{t-1})(\Delta_t'^2 - v_{t-1})} & \text{(Case 1)} \\ v_{t-1} - (1 - \beta_2) \sqrt{(v_{t-1} - \Delta_t'^2)(3\Delta_t'^2 - v_{t-1})} & \text{(Case 2)} \\ v_{t-1} - (1 - \beta_2) \Delta_t'^2 & \text{(Case 3)} \end{cases} \quad (3)$$

where Case 1, Case 2 and Case 3 represent conditions $v_{t-1} \leq \Delta_t'^2$, $\Delta_t'^2 < v_{t-1} \leq 2\Delta_t'^2$ and $v_{t-1} > 2\Delta_t'^2$ respectively. Case 1, Case 2, and Case 3 respectively correspond to: 1) relatively uniform client behavior and activity levels, where inter-client gradient differences are small (i.e., low variance); 2) moderately sparse and divergent scenarios, where user behavior starts to differ across clients; and 3) extremely sparse or highly non-IID settings, such as when some clients have almost no interactions (e.g., long-tail users), resulting in large gradient magnitude disparities across devices.

Please note that the additional server-side operations from DRA and C-SME are lightweight. DRA requires only simple element-wise operations for averaging absolute gradients, and C-SME involves conditional updates with basic arithmetic. These introduce negligible overhead compared to the cost of local training and do not affect server scalability or latency. We will explain the above three cases in Section 9. Finally, the central server updates the global model using the results of DRA and C-SME.

Divergence-Resistant Aggregation (DRA)

The motivation for divergence-resistant aggregation is alleviating extreme non-IIDness. In session-based recommendation, the models in the devices produce sparse large-scale gradients, which can magnify the performance of the aggregated model. In adaptive federated optimization methods, aggregating sparse models will significantly decrease the denominator part (line 13 of Algorithm 1), i.e., $\sqrt{v_t} + \tau = \sqrt{\beta_2 v_{t-1} + (1 - \beta_2) \Delta_t} + \tau$, where τ is a nonzero number that avoids dividing by zero. Thus, in such a case, the global model is updated in a more aggressive step size, causing fluctuated model performance. To reduce aggressive updating, we additionally compute the averaging of model gradients' absolute values as Δ'_t (line 10 of Algorithm 1), thus keeping the denominator part bigger to smooth the overall step size, which alleviates the vibration of model performance.

Although DRA addresses the sparsity-induced instability in federated model aggregation, particularly from long-tail clients whose average $|\Delta w|$ may be extremely small, it alone cannot fully guarantee training stability. In such cases, even with inflated denominators, sharp variance across devices can still lead to unstable updates. To further mitigate this, we introduce the Conditional Second Moment Estimation (C-SME) mechanism, which explicitly accounts for the update patterns based on the magnitude of $|\Delta w|$ and adjusts v_t accordingly in three well-designed cases. The next section elaborates on how the C-SME stabilizes the training even when $|\Delta w|$ is near zero or varies sharply.

Conditional Second-Moment Estimation (C-SME)

To better illustrate the motivation behind C-SME and its advantages over existing strategies, we first revisit the second-moment estimation (SME) mechanisms adopted in existing adaptive federated optimization methods. FedAdam²⁵ uses the SME formulation originally proposed in Adam⁴³:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \Delta_t'^2 = v_{t-1} - (1 - \beta_2)(v_{t-1} - \Delta_t'^2), \quad (4)$$

which performs a weighted average between the historical second moment v_{t-1} and the squared mean gradient $\Delta_t'^2$. FedYogi modifies this update with a sign operator to suppress aggressive changes:

$$v_t = v_{t-1} - (1 - \beta_2) \Delta_t'^2 \cdot \text{sign}(v_{t-1} - \Delta_t'^2). \quad (5)$$

Although both strategies aim to stabilize the learning rate by adapting the second moment, they exhibit undesirable behaviors under federated training conditions in the real world. Specifically, in highly non-IID settings with sparse or divergent client updates, both methods can cause sudden or overly aggressive changes to v_t , harming convergence stability. To systematically study this issue, we define a ratio variable V_t as:

$$V_t = \frac{v_{t-1}}{\Delta_t'^2}, \quad (6)$$

which captures the relative scale between the historical second moment and the current aggregated gradient magnitude. We also define a normalized change term:

$$\Delta \tilde{v}_t = \frac{v_t - v_{t-1}}{-(1 - \beta_2) \Delta_t'^2}, \quad (7)$$

so that $\Delta \tilde{v}_t$ can be directly interpreted as the scaled change rate of v_t .

Ideally, the value of $\Delta \tilde{v}_t$ should change smoothly with respect to V_t during training. However, Figure 3 shows that this is not the case. In FedAdam, $\Delta \tilde{v}_t$ grows linearly with V_t , leading to aggressive updates when V_t becomes large. In FedYogi, the change is bounded in magnitude, but a discontinuity occurs at $V_t = 1$, where

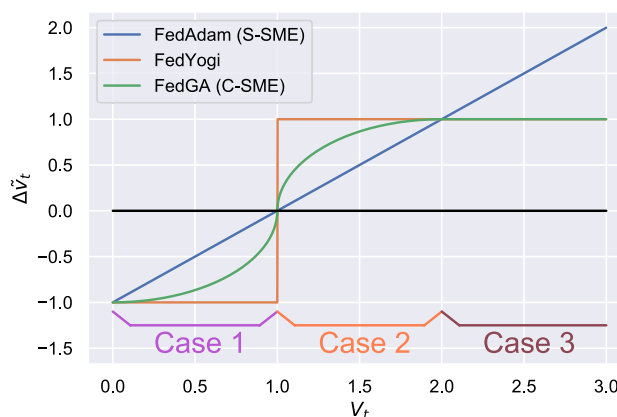


Fig. 3. Interactions between V_{t-1} and $\Delta \tilde{v}_t$.

$\Delta \tilde{v}_t$ jumps from -1 to 1 . This sudden flip introduces instability during the transition from Case 1 to Case 2. To address these limitations, we propose a continuous, case-based update mechanism in C-SME in Eq. (3). This design ensures a smooth transition across different regions and adapts to the degree of gradient divergence. The three cases in C-SME are defined as:

- **Case 1** ($v_{t-1} \leq \Delta_t'^2$): The variance of aggregated gradients is relatively high and the client behaviors are similar and dense. We apply a gentle increase to v_t to support learning without overshooting.
- **Case 2** ($\Delta_t'^2 < v_{t-1} \leq 2\Delta_t'^2$): A transitional region where gradient sparsity or divergence begins to appear across clients. We apply a smooth decay to avoid the sharp jump seen in FedYogi.
- **Case 3** ($v_{t-1} > 2\Delta_t'^2$): A highly non-IID regime where gradient magnitudes vary significantly due to client heterogeneity (e.g., long-tail users with few interactions). Conservative fixed-rate decay is used to avoid instability.

These three cases correspond to common scenarios in federated recommendation: Case 1 represents homogeneous active clients; Case 2 captures moderate divergence; and Case 3 reflects severely sparse and skewed user behavior. By tailoring the second-moment update in each case, C-SME ensures that v_t evolves continuously and smoothly throughout the training, which in turn stabilizes the effective learning rate in FedGA.

Finally, we note that the server-side computation introduced by C-SME is lightweight and involves only basic arithmetic and conditional logic. This imposes negligible overhead compared to local training or communication costs.

Theoretical analysis

Overview

The goal of this section is to establish the convergence guarantee of FedGA (Corollary 1). The key challenge in federated GNN-based recommendation lies in handling sparse and divergent gradients across clients, especially when local data distributions vary significantly. To analyze this, we start by formalizing the standard federated optimization problem and modeling the local training dynamics across devices. We make a set of standard assumptions (ie, unbiased stochastic gradients, bounded smoothness, bounded gradients, and bounded gradient variance) to allow tractable analysis. Building on these, we decompose the global gradient update into the average of local updates and then analyze how the divergence in local models affects the global optimization trajectory. The derivation focuses on bounding the norm of the gradient of the global objective, $\|\nabla f(x_t)\|^2$, over training rounds. We derive an upper bound that depends on the local update steps, the variance between clients, and the adaptive learning dynamics introduced by FedGA (notably the conditional second-moment estimation). The result shows that FedGA achieves a convergence rate of $\mathcal{O}(1/\sqrt{T})$, which matches the best-known rates for adaptive federated optimization.

Analysis details

We denote $x \in \mathbb{R}^d$ and $y \in \mathbb{R}^d$ as any two vectors in $\mathbb{R}^d$ since these results are satisfying for all $x \in \mathbb{R}^d$. Certainly, the model weights $w \in \mathbb{R}^d$ are also satisfying in these theories. First, we propose the assumptions.

Assumption 1 (Unbiased Local Gradient) The i^{th} device's local stochastic gradient $g_i(x)$ is unbiased estimation of subgradient $\nabla F_i(x)$, i.e., $\mathbb{E}_{z \sim \text{mathscr{D}}_i} [g_i(x)] = \nabla F_i(x)$.

Assumption 2 The local function F_i is L -smooth for all $i \in [m]$, i.e., $\|\nabla F_i(y) - \nabla F_i(x)\| \leq L\|x - y\|$, and $F_i(y) \leq F_i(x) + \langle \nabla F_i(x), y - x \rangle + \frac{L}{2}\|x - y\|^2$.

Assumption 3 The local objective function $f_i(x, z)$ has G -bounded gradients, that is, for any $i \in [m]$, $x \in \mathbb{R}^d$ and $z \sim \text{mathscr{D}}_i$, we have $\|\nabla f_i(x, z)\| \leq G$ ($G > 0$), for all $j \in [d]$.

Assumption 4 **Locally**, the local objective functions $f_i(x, z)$ have a σ_l -bounded variance, that is, $\mathbb{E} \left[\left| [\nabla f_i(x, z)]_j - [\nabla F_i(x)]_j \right|^2 \right] \leq \sigma_{l,j}^2$ for all $x \in \mathbb{R}^d$, $i \in [m]$, and $j \in [d]$. **Globally**, the global variance is bounded, i.e.,

$$\frac{1}{m} \sum_{i=1}^m \left[[\nabla F_i(x)]_j - [\nabla f(x)]_j \right]^2 \leq \sigma_{g,j}^2 \text{ for all } x \in \mathbb{R}^d \text{ and } j \in [d].$$

Theorem 1 Let Assumption 1 to 4 hold and with proper local stepsize $\eta_l \leq \frac{2}{KL\sqrt{43}}$. Let η_l and η be the local step size and the global step size, respectively. The iterations of FedGA satisfies

$$\begin{aligned}
\min_t \|\nabla f(x_t)\|^2 &\leq \frac{2(\sqrt{\beta_2}\eta_l KG + \tau)}{T\eta_l K} (f(x_0) - \mathbb{E}_t[f(x_T)]) \\
&+ \frac{2(\sqrt{\beta_2}\eta_l KG + \tau)}{\tau^2} \left\{ 2\eta_l L\eta_l K\sigma_l^2 + 4\sqrt{1 - \beta_2}G\eta_l K[\sigma_l^2 + \sigma_g^2 + dG^2] \right. \\
&+ 36\eta_l^3 K^3 L^4 + \sqrt{1 - \beta_2}72G\eta_l^3 K^3 L^2 + 9\eta_l^2 K^3 L^2 \tau + \eta_l LK^2 dG^2 \\
&\left. + 6\eta_l^3 K^2 L^4 + 12\sqrt{1 - \beta_2}G\eta_l^3 K^2 L^2 + \frac{3\eta_l^2 K\tau L^2}{2}\sigma^2 \right\}
\end{aligned}$$

Corollary 1 Setting $\eta_l = \min\{\frac{1}{\sqrt{T}}, \frac{2}{KL\sqrt{43}}\}$ and $\eta = \frac{1}{\sqrt{T}}$, then we have

$$\min_{0 \leq t \leq T-1} \|\nabla f(x_t)\|^2 \leq \mathit{mathscr{O}}\left(\frac{1}{\sqrt{T}}\right)$$

The above theorem and corollary provide the theoretical convergence bound of FedGA. The results indicate that FedGA obtains a convergence speed $\mathit{mathscr{O}}(1/\sqrt{T})$ similar to existing adaptive federated optimization methods.

Experiments

We conducted empirical experiments to answer the following research questions (RQ).

- **RQ1:** Can FedGA outperform the state-of-the-art adaptive FL methods on GNN-SR?
- **RQ2:** How does each strategy contribute to the performance of FedGA?

We first describe the datasets and settings and then analyze the experiment results to answer the above research questions.

Datasets

We test the performance of FedGA on four open datasets, namely Gowalla, LastFM, Yoochoose (1/4 and 1/64), and Retailrocket. Table 1 shows the statistics of the datasets mentioned above. In addition to the statistics, we also introduce a novel metric that describes the degree of non-IIDness based on the distribution of Jaccard distances between instances. Specifically, the metric is defined as

$$JD = \frac{1}{m} \frac{1}{m-1} \sum_{i=1}^m \sum_{j=1, j \neq i}^m \left(1 - \frac{|\mathit{mathscr{I}}_{(i)} \cap \mathit{mathscr{I}}_{(j)}|}{|\mathit{mathscr{I}}_{(i)} \cup \mathit{mathscr{I}}_{(j)}|}\right), \quad (8)$$

where $\mathit{mathscr{I}}_i$ is the set of items in device i . The higher JD indicates a higher degree of non-IIDness. Next, we illustrate these datasets in detail.

- **Gowalla**⁴⁴: Gowalla is a point-of-interest real-world dataset collected from a social network for users' check-in. We follow the same data processing settings as proposed in Guo et al.¹⁰ We selected the 3,000 most popular places and defined a session as a user's check-in in one day. Finally, we dropped sessions that contained more than 20 check-ins or less than 2.
- **LastFM-1K** (LastFM-1K: <http://ocelma.net/MusicRecommendationDataset/lastfm-1K.html>): LastFM is a popular music recommendation dataset that contains user clicks (user, time, artist, and song) collected between 2004 and 2009. We followed the same pre-processing as in¹³. We selected the most popular 40,000 artists and split each session as the clicks in every 8-hour interval for each user. We dropped sessions that were longer than 20 or less than 2.
- **Yoochoose** (Yoochoose dataset: <http://2015.recsyschallenge.com/challenge.html>): Yoochoose is a recommendation dataset published in the 2015 Recsys challenge that contains six-month click streams on an e-commerce website. We followed the same data processing as proposed in²⁶. We used the click streams on the last day as the validation set and selected the last 1/4 and 1/64 in the rest of the data as two training sets (Yoochoose 1/4 and 1/64). We dropped the items that are clicked with less than 5 users and sessions that contain less than 2 items.
- **Retailrocket** (Retailrocket dataset: <https://www.kaggle.com/retailrocket/ecommerce-dataset>): Retailrocket is an E-business recommendation dataset that contains 6-month click streams of users. We followed the same pre-processing procedures as proposed in¹⁴. We dropped the items that are clicked by less than 5 users and the sessions that are longer than 20 or less than 2.

We split the training data and the validation data for the above datasets from the original sessions. Specifically, an original session sequence of user i can be defined as $s_i = \{\nu_{s_i,1}, \nu_{s_i,2}, \nu_{s_i,3}, \dots, \nu_{s_i,n_i}\}$, then its training set and validation set are processed as tuples that contain a session sequence and a label, e.g., $(\{\nu_{s_i,1}\}, \nu_{s_i,2})$, $(\{\nu_{s_i,1}, \nu_{s_i,2}\}, \nu_{s_i,3})$, $(\{\nu_{s_i,1}, \nu_{s_i,2}, \nu_{s_i,3}\}, \nu_{s_i,4})$, ..., $(\{\nu_{s_i,1}, \nu_{s_i,2}, \nu_{s_i,3}, \dots, \nu_{s_i,n_i-1}\}, \nu_{s_i,n_i})$.

Dataset	#Click	#Device	#Item	#Sess.(train)	#Sess.(test)	Avg. Len	Avg. #Sess.	Max #Sess.	Min #Sess.	#Skewness	#Kurtosis	Max #Item	Min #Item	#Item Std.	JD
Gowalla	6,442,890	47,194	29,814	688,561	77,122	2.32	14.59	999	1	39.63	113.93	359	1	19.03	0.89
LastFM	3,804,922	981	36,949	1,238,523	137,646	5.70	1262.51	8,826	2	1188.08	3.36	1968	2	252.99	0.56
Yoochoose1/4	9,310,403	1,991,564	37,484	6,153,933	55,898	4.72	3.09	199	1	4.06	128.21	196	1	3.26	0.84
Yoochoose1/64	2,315,442	124,472	37,484	394,577	55,898	5.14	3.17	145	1	4.37	96.27	139	1	3.52	0.83
Retailrocket	893,945	318,697	76,061	799,930	87,512	3.11	2.51	18	1	2.79	9.20	19	1	1.72	0.94

Table 1. Statistics of session-based recommendation datasets.

Baselines and metrics

To evaluate the performance of the proposed method, we compare FedGA with the following related federated learning methods:

- **FedAvg**²³: one of the first federated learning methods that applies federated averaging to aggregate the model parameters collected from sampled devices. To date, it is still a simple but effective baseline for FL.
- **FedProx**⁴⁵: FedProx is a modified version of FedAvg that introduces the L_2 regularization term in each client's local objective, penalizing divergence from the global model.
- **FedAdam**²⁵: state-of-the-art adaptive federated optimization method inspired by adaptive optimization methods. Shows promising performance on non-IID datasets.
- **FedYogi**²⁵: a variant federated optimization method in which the SME part is defined in Eq.(5).

We validated the above federated learning methods on two GNN models for session recommendation:

- **SR-GNN**²⁶ (Session-based Recommendation with Graph Neural Networks) models session sequences as structured graph data. GNN can capture complex transitions of items based on the session graph, which are difficult to reveal by previous conventional sequential methods. Each session is then represented as the composition of the global preference and the current interest of that session using an attention neural network.
- **TA-GNN**¹⁷ (Target Attentive Graph Neural Network) captures dynamic item transitions in user sessions and generates different session embedding for each target item. It introduces a pseudo-interacted item sampling technique to protect user privacy and a graph expansion method exploiting high-order user-item interactions.

All the methods were implemented using Python 3.7 and Pytorch 2.2. We ran the experiments on two machines with 8-core 2.8-GHz Intel CPUs and 64GB memory. We tuned the best hyperparameters for the above FL and

HR@20					
Methods	Yoochoose 1/64	Yoochoose 1/4	Gowalla	LastFM	Retailrocket
SR-GNN					
SR-GNN (Centralized)	70.32 ± 0.08	71.00 ± 0.16	63.58 ± 0.02	30.66 ± 0.22	65.49 ± 0.31
FedAvg	57.74 ± 0.03	27.81 ± 0.80	21.97 ± 0.15	12.50 ± 0.03	46.38 ± 0.10
FedProx	57.32 ± 0.04	27.60 ± 0.75	21.65 ± 0.13	12.18 ± 0.06	46.14 ± 0.11
FedAdam	65.87 ± 0.27	58.55 ± 1.04	51.30 ± 0.18	27.39 ± 0.15	51.05 ± 0.17
FedYogi	69.15 ± 0.19	67.04 ± 0.24	57.71 ± 0.12	27.82 ± 0.11	61.00 ± 0.09
FedGA	69.72±0.12	67.67±0.13	58.57±0.16	27.96±0.12	61.74±0.10
TA-GNN					
TA-GNN (Centralized)	70.87 ± 0.22	71.43 ± 0.10	63.28 ± 0.05	31.12 ± 0.10	65.74 ± 0.08
FedAvg	29.51 ± 0.33	28.13 ± 0.50	22.17 ± 0.10	12.23 ± 0.05	44.29 ± 0.09
FedProx	29.46 ± 0.29	27.96 ± 0.45	21.85 ± 0.11	11.92 ± 0.07	43.95 ± 0.11
FedAdam	65.98 ± 0.25	59.37 ± 0.30	50.92 ± 0.12	27.19 ± 0.08	53.23 ± 0.14
FedYogi	68.91 ± 0.08	67.06 ± 0.12	57.38 ± 0.10	27.50 ± 0.09	61.42 ± 0.10
FedGA	69.17±0.05	67.20±0.08	58.00±0.07	27.60±0.06	61.47±0.03
MRR@20					
Methods	Yoochoose 1/64	Yoochoose 1/4	Gowalla	LastFM	Retailrocket
SR-GNN					
SR-GNN (Centralized)	30.10 ± 0.05	31.49 ± 0.15	31.93 ± 0.07	11.04 ± 0.02	41.47 ± 0.07
FedAvg	25.98 ± 0.02	17.84 ± 0.36	11.61 ± 0.05	5.76 ± 0.03	39.52 ± 0.02
FedProx	25.67 ± 0.03	17.65 ± 0.30	11.59 ± 0.06	5.60 ± 0.04	39.60 ± 0.03
FedAdam	28.00 ± 0.04	24.72 ± 0.20	24.02 ± 0.09	10.38 ± 0.05	40.02 ± 0.04
FedYogi	29.72 ± 0.04	28.46 ± 0.04	27.94 ± 0.13	10.47 ± 0.04	40.47 ± 0.05
FedGA	29.75±0.07	28.67±0.01	28.61±0.07	10.55±0.03	40.92±0.01
TA-GNN					
TA-GNN (Centralized)	30.58 ± 0.06	31.72 ± 0.10	31.80 ± 0.05	11.29 ± 0.08	41.52 ± 0.11
FedAvg	12.54 ± 0.17	17.95 ± 0.20	11.51 ± 0.05	5.80 ± 0.03	38.83 ± 0.02
FedProx	12.47 ± 0.15	17.71 ± 0.19	11.32 ± 0.06	5.51 ± 0.04	38.54 ± 0.03
FedAdam	28.07 ± 0.17	24.81 ± 0.18	24.26 ± 0.09	10.38 ± 0.05	39.36 ± 0.11
FedYogi	30.02 ± 0.17	28.53 ± 0.12	27.90 ± 0.13	10.41 ± 0.07	41.07 ± 0.13
FedGA	30.32±0.08	28.84±0.10	28.63±0.07	10.49±0.06	41.07±0.03

Table 2. Evaluation Results of FL Methods on SR-GNN and TA-GNN – Top@20 HR and MRR.

backbone GNN models by performing a grid search. We ran each experiment 3 times and reported the mean values and standard deviations of the following metrics.

- **HR@10/20** (Hit Rate): evaluates the accuracy of the model by calculating the proportion of correctly recommended items in the top 10/20 item list.
- **MRR@10/20** (Mean Reciprocal Rank): it evaluates the model ranking capability by calculating the mean reciprocal ranks of the correctly recommended items. The value of MRR is 0 when the rank exceeds 10/20.

Overall Performance (RQ1)

To answer RQ1, we compare the performance of FedGA with other baselines in session recommendation datasets. We show the results of different FL methods on SR-GNN and TA-GNN in Table 2. In summary, FedGA outperforms other baselines in all datasets and GNN models. Its performance is closer to the centralized trained GNN models compared to other FL baselines. Specifically, FedGA leads to a maximum relative improvement of 166.59% in HR @ 20 and 146.23% in MRR @ 20 over FedAvg in SR-GNN. Compared to FedYogi, FedGA improves HR@20 and MRR@20 by 1.49% and 2.4% in SR-GNN, respectively. The results show that our method performs well thanks to the proposed strategies. Although FedProx adds L_2 regularization to limit local model drift, it does not effectively reduce aggregation errors caused by client-specific embedding updates as we mentioned earlier, often performing slightly worse than FedAvg.

Figure 4 shows HR@20, MRR@20 and the test loss curves of federated learning methods. It can be seen that FedAvg fails to converge in all datasets. It should be noted that, though FedAdam achieves similar results in Table 2, FedAdam converges slower in Gowalla and Yoochoose, as shown in Figure 4. Moreover, on the two datasets with the highest JD (i.e., Retailrocket and Gowalla), FedAdam performs worse and is unstable. In Retailrocket, FedAdam experiences two performance crashes at the beginning of training and after 20,000 global epochs (see Figure 4(d), (h), and (i)). This phenomenon implies that FedAdam estimates the learning rates more aggressively, resulting in severe performance degradation. In contrast, FedYogi uses a conservative strategy to update learning rates, which makes the performance stable. Moreover, FedGA retains better accuracy and stability simultaneously by incorporating DRA and C-SME. As shown in Figure 4, FedGA reaches a comparable or better accuracy with fewer rounds, e.g., on Retailrock with high JD, FedGA reaches 60% HR@20 in less than 40,000 rounds versus more than 60,000 for FedYogi, reducing the total communication and training cost. The extra server-side computation introduced by DRA and C-SME is minimal and does not affect overall latency.

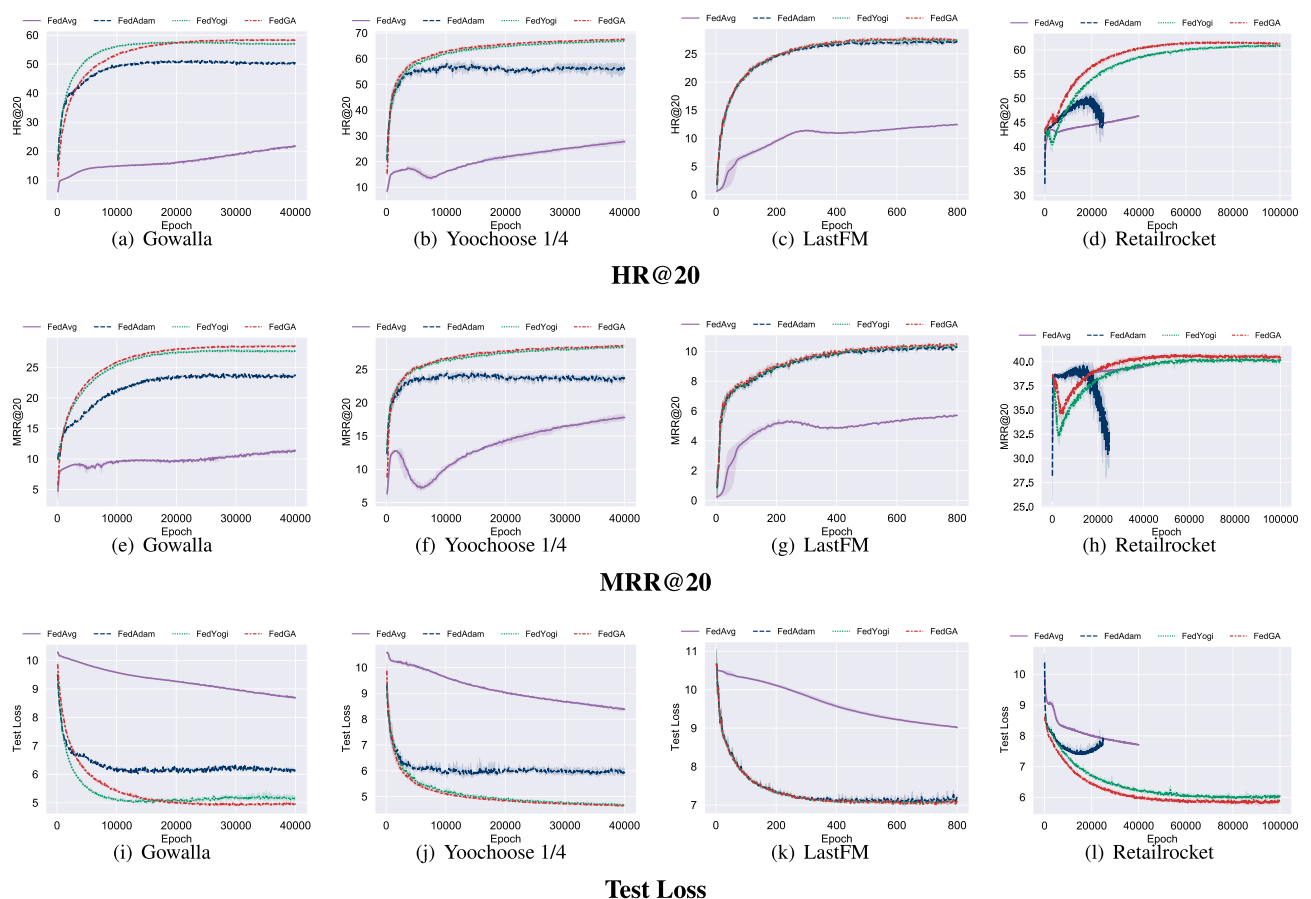


Fig. 4. Performance of FedGA on four datasets. From top to bottom, rows show HR@20, MRR@20, and test Loss.

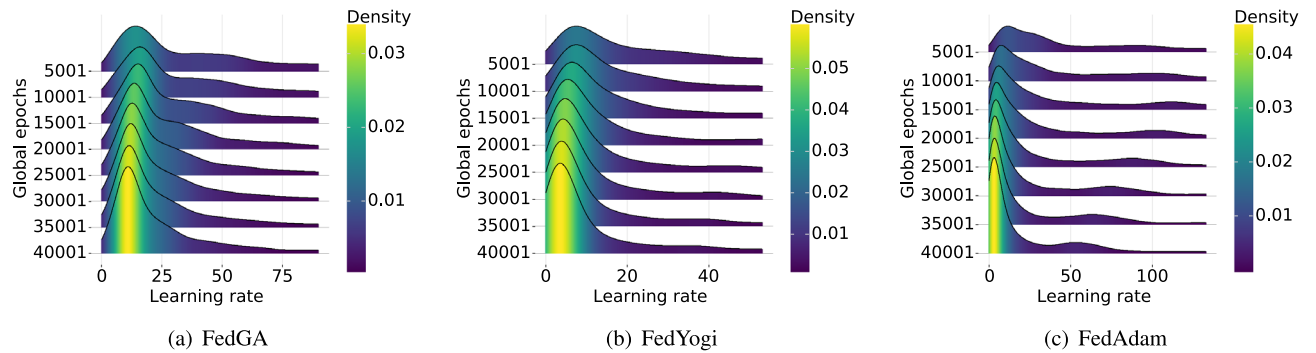


Fig. 5. Learning rates distribution in different adaptive federated learning methods.

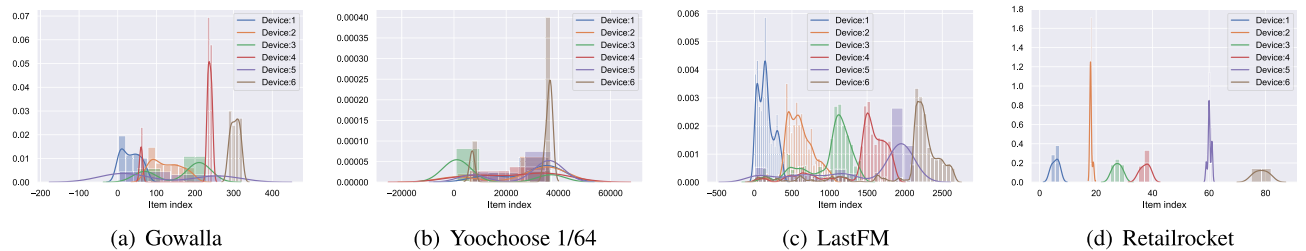


Fig. 6. Item density in different devices.

	HR@20	MRR@20	HR@10	MRR@10
(a) Effect of Local Update Steps K				
1	58.37	28.69	49.98	28.13
5	58.19	27.99	49.07	27.42
10	57.75	27.9	48.96	27.33
20	57.34	27.82	48.89	27.24
50	57.27	27.75	48.92	27.19
(b) Effect of Global Learning Rate				
1.00E-03	57.48	27.93	49.05	27.36
2.00E-03	57.4	27.76	48.83	27.18
3.00E-03	58.25	28.6	48.91	27.12
5.00E-03	58.37	28.69	49.98	28.13
0.01	57.33	27.56	48.74	27.07

Table 3. Sensitivity analysis of FedGA on the Gowalla dataset. Left: varying local update steps K ; Right: varying global learning rate η_0 .

To better understand the effects of the proposed strategies and differences between FedGA and other adaptive methods, we plot the learning rate distributions of FedGA, FedYogi, and FedAdam in Figure 5. The results clearly show that the learning rate smoothly distributes our proposal in all training epochs. Although the learning rate distributions of FedYogi and FedAdam are rather long-tailed, it implies that the variances of learning rates are higher, which may cause severe performance vibration, especially on higher-level non-IID data. In such circumstances, sessions vary on different devices (see Figure 6), leading to higher local model differences, consequently causing a more aggressive change in learning rates and, finally, affecting the stability of the global model. In FedGA, DRA and C-SME effectively control the distribution of learning rates during training, thus making the model more stable.

Finally, we evaluated the sensitivity of FedGA to two key hyperparameters: the number of local epochs K and the global learning rate η_0 . As shown in Table 3, the model exhibits only minor fluctuations in performance across a wide range of settings. For example, although the best HR@20 and MRR@20 are observed when $K = 1$, increasing K to 50 leads to less than a 2% drop in performance. Similarly, the optimal result occurs at $\eta_0 = 5 \times 10^{-3}$, but other values such as 1×10^{-3} or 1×10^{-2} still yield comparable results. These observations indicate that FedGA is robust to hyperparameter variations, retaining strong recommendation accuracy even

Methods	HR@20	MRR@20	HR@10	MRR@10
FedGA - All	58.57 ± 0.16	28.61 ± 0.07	50.14 ± 0.14	28.04 ± 0.08
C-SME (w/o DRA)	51.50 ± 0.12	24.21 ± 0.05	43.18 ± 0.13	23.60 ± 0.08
FedYogi + DRA	57.54 ± 0.05	28.01 ± 0.04	49.22 ± 0.03	27.43 ± 0.03
FedAdam + DRA	57.74 ± 0.02	28.03 ± 0.02	49.33 ± 0.04	27.44 ± 0.02
FedYogi	57.71 ± 0.12	27.94 ± 0.13	49.28 ± 0.12	27.35 ± 0.13
FedAdam (S-SME)	51.30 ± 0.18	24.02 ± 0.09	42.93 ± 0.15	23.44 ± 0.08

Table 4. Ablation study on Gowalla with SR-GNN.

when the settings are not precisely tuned. This robustness is especially valuable in real-world federated learning environments, where fine-grained tuning may be impractical.

In conclusion, the above results verify the performance of FedGA, and the effectiveness of the proposed strategies, especially for the non-IID data. In the next section, we explore the detailed contribution of each strategy and compare the proposed C-SME with the different SME strategies used in other adaptive FL baselines.

Ablation Study (RQ2)

To better understand the effectiveness and contribution of different strategies proposed in FedGA (RQ2), we performed an ablation study on C-SME and DRA in the Gowalla dataset and diagnosed these strategies. We compare the model performance of the full-version FedGA and three variants, namely C-SME (i.e., w/o DRA), FedYogi + DRA, and FedAdam + DRA. The results are reported in Table 4. It can be seen from the results that both DRA and C-SME have improved the performance of FL models differently. First, compared with FedAdam’s SME, C-SME has improved the model performance in most recommendation metrics and achieved better stability. Second, with the help of DRA, FedGA has achieved higher performance. It is worth noting that when DRA is used in FedYogi and FedAdam, the improvements are different. When DRA is used in FedAdam, the standard SME under DRA (i.e., FedAdam + DRA) has achieved a significant performance improvement (HR@20, MRR@20, HR@10, and MRR@10 have increased by 12.55%, 16.61%, 14.9%, and 1 7.06%, respectively), and C-SME greatly improved the stability of the models. In FedYogi, DRA does not directly improve HR but slightly improves MRR, and dramatically improves the stability of GNN models. This is because FedYogi’s $\Delta \tilde{v}$ throughout the processes stays stable except for the point where $V = 1$ (see Figure 3). Thus, the contribution of DRA is limited here. However, when $V = 1$, with the help of DRA, the stability of FedYogi has been dramatically improved. Although C-SME outperforms FedAdam’s SME, it seems inferior to FedYogi’s SME when it is used alone. However, after combining C-SME with DRA, the overall performance is significantly better than that of FedYogi. Such an effect implies that these two strategies are not isolated, but instead they significantly promote and interact with each other. Combining all of these strategies always leads to better performance and stability comparable to that of FedYogi.

Conclusion

This paper proposes a novel adaptive federated optimization method for session-based recommendation with GNN models. By incorporating Divergence-Resistant Aggregation (DRA) and Conditional Second-moment Estimation (C-SME), the proposed FedGA obtains efficient and stable performance over extreme non-IID data. Furthermore, we provided the theoretical convergence guarantees of FedGA and conducted empirical experiments on both open and industrial datasets to investigate FedGA’s performance. The experiment results demonstrate the effectiveness of our method. The motivation of proposing FedGA is to improve GNN’s efficiency for session recommendation. However, it is flexible to use FedGA for other neural networks and other FL use cases. Moreover, the proposed strategies can be used in conventional local adaptive optimizers, e.g., Adam, by simply modifying first- and second-moment estimations. Future directions include 1) exploring the dynamic methods for the second-moment estimation in adaptive federated optimization instead of manually chosen conditions based on the observations in FedGA, and 2) exploring how end-to-end FGL approaches can be adapted to SR scenarios. We also intend to contribute benchmark datasets specific to FL+SR scenarios (e.g., OpenFGL⁴⁶) to help foster further research in federated GNN-SR modeling.

Data availability

Datasets used in experiments can be downloaded from: LastFM-1K: <http://ocelma.net/MusicRecommendationDataset/lastfm-1K.html>, Yoochoose: <http://2015.recsyschallenge.com/challenge.html>, Retailrocket: <https://www.kaggle.com/retailrocket/ecommerce-dataset>, Gowalla: <https://snap.stanford.edu/data/loc-Gowalla.html>.

Received: 10 March 2025; Accepted: 19 June 2025
Published online: 02 July 2025

References

1. Hidasi, B., Karatzoglou, A., Baltrunas, L. & Tikk, D. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939* (2015).
2. Tan, Y. K., Xu, X. & Liu, Y. Improved recurrent neural networks for session-based recommendations. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, 17–22 (2016).

3. Hidasi, B., Quadrana, M., Karatzoglou, A. & Tikk, D. Parallel recurrent neural network architectures for feature-rich session-based recommendations. In *Proceedings of the 10th ACM conference on recommender systems*, 241–248 (2016).
4. Li, J. *et al.* Neural attentive session-based recommendation. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, 1419–1428 (2017).
5. Quadrana, M., Karatzoglou, A., Hidasi, B. & Cremonesi, P. Personalizing session-based recommendations with hierarchical recurrent neural networks. In *Proceedings of the Eleventh ACM Conference on Recommender Systems*, 130–137 (2017).
6. Jannach, D. & Ludewig, M. When recurrent neural networks meet the neighborhood for session-based recommendation. In *Proceedings of the Eleventh ACM Conference on Recommender Systems*, 306–310 (2017).
7. Hidasi, B. & Karatzoglou, A. Recurrent neural networks with top-k gains for session-based recommendations. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*, 843–852 (2018).
8. Ludewig, M. & Jannach, D. Evaluation of session-based recommendation algorithms. *User Model. User-Adapt. Interact.* **28**, 331–390 (2018).
9. Liu, Q., Zeng, Y., Mokhosi, R. & Zhang, H. Stamp: short-term attention/memory priority model for session-based recommendation. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1831–1839 (2018).
10. Guo, L. *et al.* Streaming session-based recommendation. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1569–1577 (2019).
11. Song, J. *et al.* Islf: Interest shift and latent factors combination model for session-based recommendation. In *IJCAI*, 5765–5771 (2019).
12. Ren, P. *et al.* Repeatnet: A repeat aware neural recommendation machine for session-based recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence* **33**, 4806–4813 (2019).
13. Wang, M. *et al.* A collaborative session-based recommendation approach with parallel memory modules. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 345–354 (2019).
14. Xu, C. *et al.* Graph contextualized self-attention network for session-based recommendation. In *IJCAI*, 3940–3946 (2019).
15. Qiu, R., Li, J., Huang, Z. & Yin, H. Rethinking the item order in session-based recommendation with graph neural networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, 579–588 (2019).
16. Song, W. *et al.* Session-based social recommendation via dynamic graph attention networks. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*, 555–563 (2019).
17. Yu, F. *et al.* Tagnn: Target attentive graph neural networks for session-based recommendation. *arXiv preprint arXiv:2005.02844* (2020).
18. Qiu, R., Yin, H., Huang, Z. & Chen, T. Gag: Global attributed graph neural network for streaming session-based recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 669–678 (2020).
19. Pan, Z., Cai, F., Ling, Y. & de Rijke, M. Rethinking item importance in session-based recommendation. *arXiv preprint arXiv:2005.04456* (2020).
20. Gwadabe, T. R. & Liu, Y. Improving graph neural network for session-based recommendation system via non-sequential interactions. *Neurocomputing* **468**, 111–122 (2022).
21. Sheng, Z., Zhang, T., Zhang, Y. & Gao, S. Enhanced graph neural network for session-based recommendation. *Expert Syst. Appl.* **213**, 118887 (2023).
22. Chen, Q. *et al.* Combine temporal information in session-based recommendation with graph neural networks. *Expert Syst. Appl.* **238**, 121969 (2024).
23. McMahan, B., Moore, E., Ramage, D., Hampson, S. & y Arcas, B. A. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*, 1273–1282 (PMLR, 2017).
24. Wang, S. *et al.* Adaptive federated learning in resource constrained edge computing systems. *IEEE J. Sel. Areas Commun.* **37**, 1205–1221 (2019).
25. Reddi, S. *et al.* Adaptive federated optimization. *arXiv preprint arXiv:2003.00295* (2020).
26. Wu, S. *et al.* Session-based Recommendation with Graph Neural Networks. In Hentenryck, P. V. & Zhou, Z.-H. (eds.) *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*, vol. 33 of AAAI '19, 346–353, url:10.1609/aaai.v33i01.3301346 (2019).
27. Zhao, Y. *et al.* Federated learning with non-iid data. *arXiv preprint arXiv:1806.00582* (2018).
28. Xie, M. *et al.* Multi-center federated learning. *arXiv preprint arXiv:2005.01026* (2020).
29. Briggs, C., Fan, Z. & Andras, P. Federated learning with hierarchical clustering of local updates to improve training on non-iid data. *arXiv preprint arXiv:2004.11791* (2020).
30. Yu, T., Bagdasaryan, E. & Shmatikov, V. Salvaging federated learning by local adaptation. *arXiv preprint arXiv:2002.04758* (2020).
31. Li, X. & Orabona, F. On the convergence of stochastic gradient descent with adaptive stepsizes. In *The 22nd International Conference on Artificial Intelligence and Statistics*, 983–992 (2019).
32. Sahu, A. K. *et al.* On the convergence of federated optimization in heterogeneous networks. *arXiv preprint arXiv:1812.06127* (2018).
33. Shoham, N. *et al.* Overcoming forgetting in federated learning on non-iid data. *arXiv preprint arXiv:1910.07796* (2019).
34. Jeong, E. *et al.* Communication-efficient on-device machine learning: Federated distillation and augmentation under non-iid private data. *arXiv preprint arXiv:1811.11479* (2018).
35. Cong, Y. *et al.* Fedga: A greedy approach to enhance federated learning with non-iid data. *Knowl.-Based Syst.* **301**, 112201 (2024).
36. Ju, L., Zhang, T., Toor, S. & Hellander, A. Accelerating fair federated learning: Adaptive federated adam. *IEEE Trans. Mach. Learn. Commun. Netw.* <https://doi.org/10.1109/TMLCN.2024.3423648> (2024).
37. Jiang, M., Jung, T., Karl, R. & Zhao, T. Federated dynamic gnn with secure aggregation (2020). *arXiv:2009.07351*.
38. Sajadmanesh, S. & Gatica-Perez, D. Locally private graph neural networks (2020). *arXiv:2006.05535*.
39. Zheng, L. *et al.* Asfgnn: Automated separated federated graph neural network. *ArXiv abs/2011.03248* (2020).
40. Wang, B., Li, A., Li, H. & Chen, Y. Graphfl: A federated learning framework for semi-supervised node classification on graphs. *ArXiv abs/2012.04187* (2020).
41. Wu, C., Wu, F., Cao, Y., Huang, Y. & Xie, X. Fedgnn: Federated graph neural network for privacy-preserving recommendation. *arXiv preprint arXiv:2102.04925* (2021).
42. Wan, G., Huang, W. & Ye, M. Federated graph learning under domain shift with generalizable prototypes. In *Proceedings of the AAAI conference on artificial intelligence* **38**, 15429–15437 (2024).
43. Kingma, D. P. & Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
44. Cho, E., Myers, S. A. & Leskovec, J. Friendship and mobility: User movement in location-based social networks. In *Acm Sigkdd International Conference on Knowledge Discovery & Data Mining* (2011).
45. Li, T. *et al.* Federated optimization in heterogeneous networks. *Proc. Mach. learning systems* **2**, 429–450 (2020).
46. Li, X. *et al.* Openfl: A comprehensive benchmarks for federated graph learning. *arXiv preprint arXiv:2408.16288* (2024).

Acknowledgements

This work was partially funded by the Natural Science Foundation of Zhejiang Province, China under Grant No.LTGG24F020003, and the Department of Education of Zhejiang Province, China, under Grant

No.Y202352392.

Author contributions

J.L.: Conceptualization, Methodology, Investigation, Writing – Original draft. C.R.: Supervision, Data curation. H.C.: Formal analysis, Writing – Original draft. D.L.: Supervision, Validation. All authors reviewed the manuscript.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to J.L.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

© The Author(s) 2025